

FI360 TYRE MODULE

DETAILED ENGINEERING BUILD SPECIFICATION

Fleet Intelligence 360 (FI360) — Version 1.0
Fleet Associates Africa Ltd (FAA)
10 August 2026

Document Control

Field	Value
Document ID	FI360-ENG-TYR-010
Version	1.0
Status	Engineering Build Baseline
Parent	FI360 Tyre SRS; Database/Data Dictionary; API & Integration Contract; UI/UX; Developer Backlog/Test Plan; Implementation & Deployment Blueprint
Purpose	Provide an engineering-level build specification for the FI360 Tyre module

1. Scope & Authority

This document is the engineering bridge between the approved FI360 Tyre requirements and source-code implementation. It defines the proposed engineering organization, domain boundaries, API resource catalogue, database relationship model, state transitions, validation, events, frontend routes/components and implementation sequence.

Where the approved parent documents specify exact field names, payloads, constraints or contracts, those documents remain authoritative. This document should be treated as an implementation blueprint and any discrepancy must be resolved through controlled change management rather than silently altering the approved requirements.

2. Engineering Architecture

User → Tyre Web → API Gateway/Service → Domain Service → Repository → PostgreSQL

Request path

Committed Transaction → Domain Event → Event Bus → Subscribed FI360 Module → Integration Handler

Integration path

Boundary	Responsibility
Tyre Web	User interaction and presentation
Tyre API	External interface and request validation
Application Layer	Use-case orchestration
Domain Layer	Tyre business rules/state transitions
Persistence Layer	Database access
Integration Layer	Events and optional module adapters
Core Platform	Identity, tenant, authorization, shared UI and platform services

3. Domain Model

Aggregate/Entity	Purpose	Key Relationships
Tyre	Master identity and current lifecycle	Inspections, fitments, movements,

	state	lifecycle events
Asset Reference	Canonical asset context when available	Tyre fitments/positions
Tyre Position	Position within an asset	Fitment history
Inspection	Measured condition at a point in time	Tyre, asset/position context
Stock Location	Physical storage context	Tyre movements
Movement	Traceable stock/location transaction	Tyre, from/to location
Fitment	Association of tyre to asset position	Tyre, asset, position
Repair	Repair intervention	Tyre
Retread	Retread cycle/intervention	Tyre/casing
Disposition	Retirement/scrap outcome	Tyre
Lifecycle Event	Chronological history	Tyre
Audit Record	Security/business audit trail	User, tenant, transaction

4. Tyre State Model

REGISTERED → IN STOCK → FITTED → INSPECTED → REMOVED → REPAIR/RETREAD → RETURN TO STOCK/FIT → RETIRED/SCRAPPED

Core lifecycle

The exact status vocabulary must follow the approved data dictionary/API contract. The engineering implementation should prevent invalid transitions and preserve every material transition as history.

Transition	Required Controls
Register → In Stock	Identity uniqueness, required master data
In Stock → Fitted	Eligibility, asset/position validation
Fitted → Inspected	Tyre exists and is active
Fitted → Removed	Reason, condition and destination
Removed → Repair	Valid lifecycle action
Repair → In Stock/Fitted	Result and completion recorded
Retread → In Stock/Fitted	Cycle/result recorded
Any eligible state → Retired	Authorized reason/approval
Retired → Scrap/Disposition	Controlled final disposition

5. Database Relationship Blueprint

Table/Concept	Primary Key	Core Foreign Keys
tyres	tyre_id	tenant_id, specification_id, location_id
tyre_specifications	specification_id	tenant_id
tyre_fitments	fitment_id	tenant_id, tyre_id, asset_id, position_id
tyre_inspections	inspection_id	tenant_id, tyre_id, fitment_id
tyre_locations	location_id	tenant_id, parent_location_id
tyre_movements	movement_id	tenant_id, tyre_id, from_location_id, to_location_id
tyre_repairs	repair_id	tenant_id, tyre_id
tyre_retreads	retread_id	tenant_id, tyre_id
tyre_dispositions	disposition_id	tenant_id, tyre_id
tyre_lifecycle_events	event_id	tenant_id, tyre_id
tyre_audit_records	audit_id	tenant_id, tyre_id, actor_id

The exact physical table names, columns and constraints must be reconciled against the approved Database/Data Dictionary before implementation. This section intentionally defines the engineering relationship pattern rather than inventing unapproved column-level requirements.

6. Database Integrity Rules

- All tenant-owned records carry tenant context.
- Primary keys must be stable and non-recycled.
- Foreign keys protect relationship integrity.
- Unique constraints prevent duplicate identifiers according to approved business rules.
- Material lifecycle records should be append-oriented.
- Use timestamps consistently and store timezone-aware values where required.
- Use database transactions for multi-record operational commands.
- Index high-frequency search keys and tenant-scoped operational queries.
- Do not use hard deletion for material lifecycle history unless explicitly approved.

7. API Resource Catalogue

Resource	Primary Operations
/api/v1/tyres	List, create, retrieve, update approved master attributes
/api/v1/tyres/{id}	Retrieve tyre details/history
/api/v1/tyres/{id}/fitments	Fitment history/current fitment
/api/v1/tyres/{id}/inspections	Inspection history/create inspection
/api/v1/tyres/{id}/movements	Stock/location movements
/api/v1/tyres/{id}/repairs	Repair records
/api/v1/tyres/{id}/retreads	Retread records
/api/v1/tyres/{id}/dispositions	Disposition records
/api/v1/dashboard	Tyre KPI/exception summary
/api/v1/reports	Approved report catalogue/query
/api/v1/alerts	Tyre alerts and exception workflow

These resource paths are an engineering organization proposal. Before coding, align exact paths, verbs, request/response schemas and error codes with the approved API & Integration Contract.

8. Command vs Query Pattern

Type	Examples
Commands	RegisterTyre, ReceiveTyre, MoveTyre, FitTyre, RemoveTyre, InspectTyre, RotateTyres, RecordRepair, RecordRetread, RetireTyre
Queries	GetTyre, SearchTyres, GetTyreHistory, GetStock, GetInspectionHistory, GetDashboard, GetReports
Integration Commands	LinkWorkOrder, AssociateAsset, SyncOperationalReading — only when enabled
Events	TyreRegistered, TyreReceived, TyreFitted, TyreRemoved, TyreInspected, TyreRepaired, TyreRetreaded, TyreRetired

9. API Request Processing

Authenticate → Resolve Tenant → Authorize → Validate DTO → Load Aggregate → Apply Domain Rule → Persist Transaction → Write Audit → Publish Event → Return Response

Command pipeline

- Authorization must happen before exposing protected data.
- Domain validation must not rely only on frontend validation.
- Transaction commit must precede reliable event publication; use an approved outbox pattern if required.
- Return correlation/reference identifiers for traceability.
- Do not expose internal stack traces to clients.

10. Standard API Response Model

Response Type	Recommended Shape
Success	data + metadata/correlation reference
List	items + pagination metadata
Validation Error	standard error code + field errors
Authorization Error	standard access-denied response
Not Found	resource-not-found code
Conflict	business/state conflict code
Integration Pending	local transaction result + integration status where applicable
Server Error	generic safe error + correlation reference

Exact property names must follow the approved API contract.

11. Validation Catalogue

Area	Validation
Identity	Required identifier, uniqueness according to policy
Specification	Valid controlled specification
Stock	Valid location and movement rules
Fitment	Eligible tyre, valid asset and position
Inspection	Valid units/ranges and required checks
Removal	Required reason/destination
Repair	Required intervention data
Retread	Cycle/supplier/result according to approved rules
Disposition	Authorization/reason and final-state rules
Permissions	Operation permitted for role/user

12. Idempotency & Concurrency

- Commands that may be retried must support an idempotency mechanism where appropriate.
- Do not allow duplicate fitment or removal from repeated requests.
- Use optimistic concurrency/version checks where simultaneous edits are possible.
- Use database transaction boundaries for state-changing commands.
- Return conflict responses when the requested state is no longer valid.

13. Frontend Route Map

Route	Screen
/tyre	Tyre Dashboard
/tyre/register	Register Tyre
/tyre/tyres	Tyre Register/Search
/tyre/tyres/{id}	Tyre Details
/tyre/tyres/{id}/fit	Fitment
/tyre/tyres/{id}/inspect	Inspection
/tyre/tyres/{id}/remove	Removal
/tyre/stock	Stock & Location
/tyre/operations/rotation	Rotation
/tyre/lifecycle/repair	Repair
/tyre/lifecycle/retread	Retread
/tyre/lifecycle/disposition	Disposition
/tyre/alerts	Alerts
/tyre/reports	Reports

14. Frontend Component Map

Component	Purpose
TyreSearchBar	Identifier/filter search
TyreTable	Paged register
TyreStatusBadge	State display
TyreSummaryCard	Core tyre context
TyreTimeline	Lifecycle history
FitmentForm	Asset/position association
InspectionForm	Measurements/condition
RemovalForm	Removal workflow
StockMovementForm	Location transaction
LifecycleForm	Repair/retread/disposition
AlertCard	Exception/action
KpiCard	Dashboard metric
ConfirmTransactionDialog	Material action confirmation

15. UI-to-API-to-Database Mapping

UI	API Use Case	Persistence
Tyre ID	Create/Get Tyre	tyres
Specification	Create/Get Tyre	tyre_specifications
Location	Receive/Move	tyre_locations + tyre_movements
Asset/Position	Fit/Remove	tyre_fitments
Tread/Pressure	Inspect	tyre_inspections
Repair details	Record Repair	tyre_repairs
Retread details	Record Retread	tyre_retreads
Disposition	Retire/Scrap	tyre_dispositions
Timeline	Get History	tyre_lifecycle_events
Actor/action	Audit	tyre_audit_records

16. State Transition Service

Implement a single domain-level transition mechanism rather than allowing controllers or UI components to directly change status fields.

- Load current aggregate/state.
- Validate requested transition.
- Validate permissions.
- Validate required supporting data.
- Persist state and history atomically.
- Create audit entry.
- Prepare/publish integration event.
- Return resulting state.

17. Fitment Transaction

Select Tyre → Select Asset → Select Position → Validate → Begin Transaction → End Existing Conflict → Create Fitment → Update Tyre State → Audit → Commit → Publish Event

Fitment command

- Prevent two active fitments for the same tyre.
- Prevent conflicting active tyre assignments to the same position unless an approved replacement workflow is being executed.
- Capture usage context where required.

- Return fitment reference and resulting tyre/asset state.

18. Inspection Transaction

Load Tyre → Validate Inspector → Capture Measurements → Validate Range/Unit → Evaluate Rules → Persist Inspection → Update Alert State → Audit → Return Result

Inspection command

Inspection evaluation rules should be configurable where the approved requirements permit configuration, rather than hard-coding every threshold into the frontend.

19. Removal Transaction

Load Active Fitment → Validate Reason → Capture Condition/Usage → Close Fitment → Set Destination → Update State → Audit → Publish Event

Removal command

Removal must not simply delete the current fitment; it must close the relationship and preserve history.

20. Stock Movement Transaction

Load Tyre → Validate Current Location → Validate Destination → Create Movement → Update Current Location → Audit → Publish Event

Movement command

- Reject movement when the source location is no longer current.
- Support controlled reconciliation where approved.
- Do not create duplicate movements from retried requests.

21. Lifecycle Transaction Services

Service	Core Responsibility
RepairService	Validate and record repair intervention
RetreadService	Manage retread cycle and result
DispositionService	Manage retirement/scrap state
LifecycleHistoryService	Maintain chronological event representation
CostService	Calculate approved lifecycle cost metrics

22. Event Contract Structure

Field	Purpose
event_id	Unique event reference
event_type	Controlled event name
event_version	Contract version
occurred_at	Event time
tenant_id	Tenant context
actor_id	Originating user/service
aggregate_type	Tyre
aggregate_id	Tyre identifier
correlation_id	Cross-service tracing
payload	Approved event-specific data

Exact event payloads must be taken from the approved API/Event Contract when available. Do not invent additional mandatory payload fields without change control.

23. Optional Workshop Integration

Tyre Inspection Exception → Integration Adapter → Create/Link Work Order → Workshop Executes → Repair Result → Tyre Lifecycle Updated

Tyre ↔ Workshop

- Integration must be optional.
- Tyre remains operational if Workshop is unavailable.
- Use stable identifiers and correlation IDs.
- Do not duplicate Workshop's work-order source of truth.
- Handle integration failure explicitly.

24. Optional Fleet/Telematics Integration

Fleet/Telematics → Canonical Asset/Mileage Event → Tyre Service → Usage/Performance Analytics

Operational context

If Fleet or Telematics is not installed, Tyre must still support approved manual or alternative operational readings. When integration is enabled, external data should enrich rather than break core Tyre workflows.

25. Authorization Matrix

Operation	Permission
View tyre	tyre.read
Register	tyre.create
Receive/move	tyre.receive / tyre.move
Fit	tyre.fit
Remove	tyre.remove
Inspect	tyre.inspect
Rotate	tyre.rotate
Repair	tyre.repair
Retread	tyre.retread
Disposition	tyre.dispose
Cost view	tyre.cost
Approve	tyre.approve
Admin	tyre.admin

Exact permission identifiers must align with the FI360 Core authorization model before implementation.

26. Error Handling

Condition	Expected Handling
Invalid input	422-style validation response or approved equivalent
Unauthenticated	401-style response or platform equivalent
Unauthorized	403-style response or platform equivalent
Missing resource	404-style response
State conflict	409-style response
Integration failure	Explicit integration status; no false success
Unexpected error	Safe generic response + correlation ID

27. Testing Hooks

- Every state-changing service should be unit-testable without HTTP.
- Repositories should be mockable/testable.
- API contract tests should validate schemas.
- Event publication should be testable separately from event consumers.
- Feature flags should be controllable in test environments.
- Seed data should represent realistic lifecycle scenarios.

28. Seed/Test Data

Dataset	Purpose
TYR-DEMO-01	New tyres in stock
TYR-DEMO-02	Tyres currently fitted
TYR-DEMO-03	Tyres with inspection history
TYR-DEMO-04	Tyres with repair history
TYR-DEMO-05	Tyres with retread history
TYR-DEMO-06	Retired/scrapped tyres
TYR-DEMO-07	Exception/alert cases
TYR-DEMO-08	Long lifecycle history for performance tests

29. Engineering Sequence

1. Freeze parent-document baselines and traceability IDs.
2. Create repository/package structure.
3. Implement Core identity/tenant/authorization adapters.
4. Create database migration 001 and reference data.
5. Implement Tyre aggregate/domain services.
6. Implement Tyre master/search API.
7. Implement Tyre register/details UI.
8. Implement stock/location services and UI.
9. Implement fitment/removal transactions.
10. Implement inspection and alert rules.
11. Implement repair/retread/disposition.
12. Implement dashboard/report queries.
13. Implement event/outbox/integration adapters.
14. Execute automated tests and security tests.
15. Deploy to QA/UAT.
16. Close defects and release.

30. Build Acceptance Gates

Gate	Evidence
Engineering Ready	Repository, standards, CI/CD ready
Data Ready	Migration + constraints + seed validated
API Ready	Contract tests pass
UI Ready	P0 workflows functional
Domain Ready	State transitions tested
Integration Ready	Events/adapters tested
Security Ready	Authorization/tenant tests pass
Release Ready	UAT + production gates pass

31. Code Quality Standards

- Strong TypeScript typing.
- No business logic in UI-only components.
- No direct database access from controllers/frontend.
- Explicit DTO/schema validation.
- Consistent error handling.
- Automated lint/type/test gates.
- Code review for every production change.
- Meaningful names over excessive abstraction.
- Document non-obvious business rules.
- Keep module dependencies intentional and one-directional.

32. Engineering Anti-Patterns to Avoid

- Tyre UI directly manipulating database records.
- Duplicating Fleet or Workshop master data as a second source of truth.
- Hard-coding integration dependencies.
- Status changes without lifecycle history.
- Authorization only in frontend.
- Unversioned APIs.
- Large synchronous report queries blocking operational APIs.
- Untracked manual production database edits.
- Publishing events before transaction commit without a reliable delivery pattern.

33. Build Deliverables

Deliverable	Output
Source	Tyre Web + Tyre API
Database	Migrations + seed + indexes
API	Versioned contract implementation
Events	Approved Tyre events
UI	P0/P1 screens
Tests	Automated + system + security
Infrastructure	Containers + deployment manifests
Documentation	Developer setup + operations + API docs
Release	Versioned artifact + release notes

34. Engineering Handoff

At the end of this document, the development team has a controlled blueprint for the implementation structure. The remaining work is to reconcile the exact database fields and API payloads against the already-approved parent documents before code generation.

The next logical document should therefore focus on the final executable engineering artefacts rather than another conceptual description.

35. Next Document — Document 11

DOCUMENT 11 should be the FI360 TYRE MODULE DATABASE SCHEMA, API ENDPOINT & CONTRACT CATALOGUE. It should contain the final field-by-field database schema, data types, nullability, indexes, foreign keys, API verbs, exact request/response payloads, validation rules, error codes, pagination, filtering, event schemas and example transactions. It should be the principal reference for backend and frontend developers.

F1360 engineering principle: one source of truth for each domain, stable contracts between modules.